Overlapping Cooperative Co-Evolution for Overlapping Large-Scale Global Optimization Problems

Marcin M. Komarnicki
Department of Systems and Computer Networks
Wroclaw University of Science and Technology
Wroclaw, Poland
marcin.komarnicki@pwr.edu.pl

Michal W. Przewozniczek
Department of Systems and Computer Networks
Wroclaw University of Science and Technology
Wroclaw, Poland
michal.przewozniczek@pwr.edu.pl

Renato Tinós
Department of Computing and Mathematics
University of São Paulo
Ribeirão Preto, São Paulo, Brazil
rtinos@ffclrp.usp.br

Xiaodong Li
School of Computing Technologies
RMIT University
Melbourne, Australia
xiaodong.li@rmit.edu.au

ABSTRACT

One of the main approaches for solving Large-Scale Global Optimization (LSGO) problems is embedding a decomposition strategy into a Cooperative Co-Evolution (CC) framework. Decomposing an LSGO problem into smaller subproblems and optimizing them separately using a CC framework was shown to be effective when a considered problem is partially separable. Components in CC frameworks are usually disjoint. Thus, the existence of the perfect decomposition of such problems allows of the optimization of independent components. However, for overlapping problems, the perfect, unique decomposition does not exist due to the existence of shared variables. Despite this, each variable is usually assigned to a single component, and the assignment does not change during a whole framework run. In this paper, we propose a new CC framework that allows multiple assignments of shared variables. Allocating computational resources to each of its components is influenced by other components that share variables with it. According to experimental results, our proposed method outperforms the state-of-the-art LSGO-dedicated optimization methods, including other CC frameworks, when overlapping LSGO problems are considered.

CCS CONCEPTS

• **Computing methodologies** → **Artificial intelligence**; **Continuous space search**.

KEYWORDS

Cooperative Co-Evolution, Large-Scale Global Optimization, Overlapping problem

ACM Reference Format:
Marcin M. Komarnicki, Michal W. Przewozniczek, Renato Tinós, and Xiaodong Li. 2024. Overlapping Cooperative Co-Evolution for Overlapping Large-Scale Global Optimization Problems. In *Genetic and Evolutionary Computation Conference (GECCO '24), July 14–18, 2024, Melbourne, VIC, Australia.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3638529.3654171

GECCO '24, July 14–18, 2024, Melbourne, VIC, Australia

ACM ISBN 979-8-4007-0494-9/24/07.
https://doi.org/10.1145/3638529.3654171

1 INTRODUCTION

Solving Large-Scale Global Optimization (LSGO) problems is important in many scientific and engineering fields [3, 6, 9]. In this paper, we consider real-value encoded instances. One of the main approaches to solve these *complex continuous optimization problems* [30] consists of two phases. A problem is first decomposed into smaller subproblems [2, 11, 15, 19, 25]. Then, subproblems are optimized separately. To this end, a Cooperative Co-Evolution (CC) framework is utilized [7, 21, 23, 29]. However, a high-quality problem decomposition is frequently required to make this approach effective [11]. In the case of partially separable problems, they can be divided into smaller subproblems that are pairwise independent. For overlapping problems, such a decomposition is impossible [22].

Assigning overlapping problem variables to components that will be optimized separately is a challenging task [7, 26]. Most state-of-the-art LSGO-dedicated CC frameworks assume that one variable can be a member of only one component. Thus, at least one component contains variables that are dependent on variables from outside this component, or all variables are joined into one large component, which does not reduce the problem size. In this paper, we propose a new CC framework, namely Overlapping Cooperative Co-Evolution (OCC). In OCC, each variable can belong to many components. Consequently, it is possible to create a dedicated component for each subproblem. Thus, components may exactly reflect an underlying problem structure. OCC like most CC frameworks, tries to update the best solution found so far after the component optimization. Due to shared variables, it may be unsuccessful in many attempts. To not neglect some components, we propose a new mechanism to allocate computational resources to components based on the successful attempts of other components that share variables with them. The main objective of this paper is to show that incorporating overlapping components into a CC framework improves its effectiveness for overlapping LSGO problems.

The rest of this paper is organized as follows. A discussion about the related work is included in the next section. In Section 3, we

present OCC thoroughly. Section 4 presents the results of the conducted experiments and their analysis. Finally, the paper is concluded in the last section.

2 RELATED WORK

In this section, we provide two definitions of the problem separability and present decomposition strategies first. Secondly, CC frameworks are described. Finally, another main approach to tackle LSGO problems, namely hybrid optimization methods, is presented.

2.1 Problem decomposition

The decomposition of a problem depends on the separability structure of its objective function [22]. The most common definition of the function separability is as follows [11, 15, 19, 22, 29]. Let $f : \Omega \rightarrow \mathbb{R}$ be a n-dimensional function. It is partially separable [20] and consists of m separable subfunctions if

$$\arg\min_{\boldsymbol{x}\in\Omega} f(\boldsymbol{x}) = \left[\arg\min_{\boldsymbol{x_1}\in\Omega_1} f(\boldsymbol{x_1}, ...), ..., \arg\min_{\boldsymbol{x_m}\in\Omega_m} f(..., \boldsymbol{x_m}) \right] \quad (1)$$

where $2 \leq n \leq m$, $\boldsymbol{x}$ is a decision vector $[x_1, ..., x_n]$, and $\Omega = \Omega_1 \times ... \times \Omega_m$. However, this definition does not always take fitness landscape characteristics [16] into account [11]. Alternatively, a definition provided in [24] takes into account a possible significant influence of fitness landscape characteristics on the optimizer effectiveness [4, 10] and is as follows. Let X_1 and X_2 be two disjoint sets of decision variables. They do not interact if conditions

$$f(\hat{\boldsymbol{x}}) < f(\hat{\boldsymbol{x}} + \delta_1\boldsymbol{u_1}) \Leftrightarrow f(\hat{\boldsymbol{x}} + \delta_2\boldsymbol{u_2}) < f(\hat{\boldsymbol{x}} + \delta_1\boldsymbol{u_1} + \delta_2\boldsymbol{u_2}) \quad (2)$$

and

$$f(\hat{\boldsymbol{x}}) = f(\hat{\boldsymbol{x}} + \delta_1\boldsymbol{u_1}) \Leftrightarrow f(\hat{\boldsymbol{x}} + \delta_2\boldsymbol{u_2}) = f(\hat{\boldsymbol{x}} + \delta_1\boldsymbol{u_1} + \delta_2\boldsymbol{u_2}) \quad (3)$$

hold for any decision vector $\hat{\boldsymbol{x}} \in \Omega$, values $\delta_1 > 0$, $\delta_2 > 0$, and unit vectors $\boldsymbol{u_1} \in U_{X_1}$, $\boldsymbol{u_2} \in U_{X_2}$ such that they form a feasible solution of f [11, 24]. Unit vector $\boldsymbol{u_j} = [u_1, ..., u_n]$ belongs to U_{X_j} if $\forall_{i\in 1,...,n} u_i = 0 \Leftrightarrow x_i \notin X_j$.

An underlying structure of a considered optimization problem is not explicitly given in black-box optimization. However, decomposition strategies can discover dependencies between decision variables. When a considered optimization problem consists of additively separable subproblems, decomposition strategies derived from Differential Grouping (DG) [19] can provide the high-quality decomposition [15, 22, 25]. Otherwise, more general decomposition strategies are useful [2, 11]. The interaction matrix Θ may be an output of decomposition strategies. Values of its elements $\theta_{p,q}$, 1 or 0, indicate if two variables x_p and x_q interact or not. The interaction matrix can be considered as a variable interaction graph (VIG) [28], in which vertices are related to variables and an edge between the pth and qth vertices exists only when $\theta_{p,q} = 1$. Then, groups of interacting variables are connected components of VIG [22]. In case of overlapping problems, i.e., some of their variables are shared among several subproblems, finding the connected components often leads to grouping all variables into one group despite the existence of pairs of variables that do not interact with each other. Nevertheless, joining variables into smaller groups may be beneficial for overlapping problems with conforming subproblems [11, 26]. Therefore, in Recursive Differential Grouping 3 (RDG3) [26] a user-defined threshold ϵ_n was introduced. Variables of two overlapping subproblems should not be grouped together if the size of at least one subproblem is greater than or equal to ϵ_n.

2.2 Cooperative Co-Evolution

Reducing the problem size is the key idea behind Cooperative Co-Evolution (CC) frameworks [7, 21, 23, 29]. Embedding a decomposition strategy into a CC framework may result in creating components that reflect subproblems of a considered problem. In CC, components are optimized separately. Thus, a combination of a decomposition strategy and a CC framework is very effective for partially separable problems [11, 15, 26]. To improve the efficiency of CC frameworks, optimizing components in a round-robin fashion (one after another) is replaced by selecting the next component to be optimized such that its contribution on global fitness improvements is the highest [21, 26, 29]. Such frameworks are denoted as Contribution-based Cooperative Co-Evolution (CBCC) frameworks [21, 29].

In CC, components are usually disjoint, and the variable assignments usually do not change during a CC run. When a CC framework uses groups of variables provided by a decomposition strategy, decisions on the assignment of each shared variable (to one of its related components) are actually made by the decomposition strategy. However, a CC framework can analyze the interaction matrix provided by a decomposition strategy on its own. CBCCO [7] is a recent proposition of a CC framework that creates components consisting of interacting non-shared variables and a set of shared variables based on the given interaction matrix. Then, each component is optimized separately in a round-robin fashion for a user-defined number of iterations. During this phase, an accumulated contribution of each component is calculated based on global fitness improvements. Shared variables are then assigned to the best related component, i.e., a related component that has achieved the highest value of an accumulated contribution. When all variables are already assigned, CBCOO still optimizes components one after another and calculates accumulated contributions. At the end of each CBCOO iteration, accumulated contributions are used to create a list of components that should be awarded and optimized again.

2.3 Hybrid optimization methods

Another effective approach to solve LSGO problems is using different optimization methods during the same optimization process [12, 17]. The optimization methods optimize all decision variables together or their random subset. The participation of each optimizer is usually proportional to its contribution to global fitness improvements. An example of the state-of-the-art hybrid optimization method is an extension of Success-History Based Parameter Adaptation for Differential Evolution (SHADE) [27] namely SHADE with Iterative Local Search (SHADE-ILS) [17]. SHADE-ILS employs SHADE to explore the search space globally. SHADE is executed alternately with one of two local search methods, MTS-L1 [14] and L-BFGS-B [18], depending on the contribution of each one. They have different strengths and weaknesses. Fully separable problems are suitable for MTS-L1, whereas L-BFGS-B is more robust. Since L-BFGS-B is a gradient-based optimization method, the gradient

must be approximated in black-box optimization. SHADE-ILS also detects when it should be restarted due to the low improvement ratio.

3 PROPOSED COOPERATIVE CO-EVOLUTION FRAMEWORK

When a considered optimization problem consists of overlapping subproblems, CC frameworks either optimize all overlapping subproblems within one component or assign shared variables to one of their related components. Optimizing one large component does not reduce the problem size. On the other hand, the assignment of shared variables only to one component affects the optimization process of the other related components. Moreover, a CC framework may be trapped in a local optimum in case of inappropriate assignments. Therefore, we propose an Overlapping Cooperative Co-Evolution (OCC) framework. Due to multiple assignments of the same variable and a new allocation computational resources mechanism, OCC can effectively solve overlapping LSGO problems.

3.1 Computational resource allocation

Allowing of multiple assignments of shared variables results in some difficulties. In typical CC frameworks, components are disjoint. Since the best solution found so far is usually used as the context solution, improvements during the component optimization cause finding the new best solution. In case of overlapping components, better component solutions in comparison to previous iterations may not fit into the best solution found so far because the component share variables can be outdated. Latest improvements of the best solution found so far made by related components can significantly increase the distance between values of the component shared variables when we compare the best solution found so far with the previous best solution found by the component. Let us consider the following example. In Figure 1, we present the VIG of a function $\tilde{f}$ that can be defined as $\tilde{f}(\boldsymbol{x}) = \tilde{f}_1(x_1, x_2, x_3, x_4) + \tilde{f}_2(x_4, x_5, x_6, x_7) + \tilde{f}_3(x_7, x_8, x_9, x_{10})$. This problem consists of three subproblems. Let X_i be a set of variables belonging to the ith subproblem. Then, $X_1 = \{x_1, x_2, x_3, x_4\}$, $X_2 = \{x_4, x_5, x_6, x_7\}$, and $X_3 = \{x_7, x_8, x_9, x_{10}\}$. Shared variables are variables that belong to more than one subproblem, e.g., x_4 is a member of X_1 and X_2. Let $\tilde{x}_{init} = [a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8, a_9, a_{10}]$ be an initial solution and CC components reflect subproblems exactly, e.g., the first component consists of four variables: x_1, x_2, x_3, and x_4. After optimizing subproblem $\tilde{f}_2$, the best solution found so far is $\tilde{x}_{b1} = [a_1, a_2, a_3, \mathbf{b_4}, \mathbf{b_5}, \mathbf{b_6}, \mathbf{b_7}, a_8, a_9, a_{10}]$. Then, the optimization processes of subproblems $\tilde{f}_1$ and $\tilde{f}_3$ result in finding the new best solutions $\tilde{x}_{b2} = [\mathbf{c_1}, \mathbf{c_2}, \mathbf{c_3}, \mathbf{c_4}, b_5, b_6, b_7, a_8, a_9, a_{10}]$ and $\tilde{x}_{b3} = [c_1, c_2, c_3, c_4, b_5, b_6, \mathbf{d_7}, \mathbf{d_8}, \mathbf{d_9}, \mathbf{d_{10}}]$, respectively. The next subproblem to optimize is $\tilde{f}_2$ again. The component optimizer remembers that it internal best solution found so far is $[x_4, x_5, x_6, x_7] = [b_4, b_5, b_6, b_7]$. Note that the values of x_4 and x_7 (they are shared variables) are different when compared with values from the best solution found so far by the CC framework, i.e., $x_4 = c_4$ and $x_7 = d_7$. If values of x_4 and x_7 influence the fitness of $\tilde{f}_1$ and $\tilde{f}_3$ the most and the contribution of these two subproblems to the fitness of the considered problem is significant, the component optimizer may not be able to find the new best solution until values of the shared variables approach values from the best solution found so far by the CC framework. Moreover, even if subproblem $\tilde{f}_2$ influences the fitness of the considered problem the least, injecting values of its non-shared variables (x_5 and x_6) into the best solution found so far by the CC framework may be required to prevent premature convergence of the optimization processes of the other subproblems.

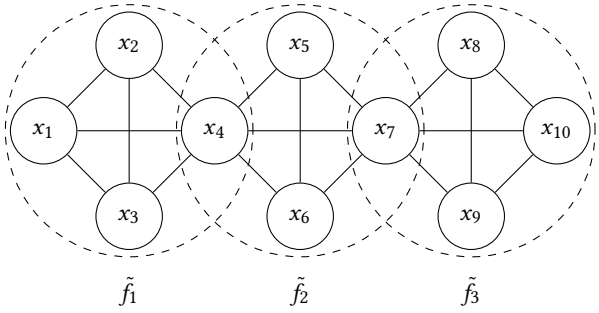


Figure 1: VIG of the exemplary problem that consists of three subproblems.

Our proposition does not suffer from such a drawback. In OCC, components that have updated the best solution increase the number of iterations of the optimization processes of their adjacent (related) components during the next OCC cycle. A component is adjacent to another component if they share at least one variable. Furthermore, components that already found the best solution decrease the number of iterations of their component optimizers in the next OCC cycle. Such a mechanism should increase the probability that values of shared variables approach values from the best solution found so far or more promising search region is found before repeated finding the new best solution by related components. Note that it is a general mechanism of allocating computational resources to components that is suitable for any optimizer. We would not like to inject the best values found so far of shared variables to solutions or models of component optimizers, because then we could interrupt some of their internal mechanisms, e.g., adaptation of optimizer's parameters.

3.2 Overlapping Cooperative Co-Evolution

The general procedure of OCC is based on the standard CC [23, 26] and is presented in Pseudocode 1. OCC needs the interaction matrix to create overlapping components (Pseudocode 1: line 1) that fully cover all subproblems. Thus, for each decision variable, it is required to know to which subproblems the variable belongs. To this end, we can apply the grouping method proposed in [8] using the interaction matrix (which is provided to OCC), and the acceptable overlapping rate equals to 1 as its inputs. When the acceptable overlapping rate is set to 1, none of two groups are merged even if they share most of their variables. For each detected subproblem, we create a component that covers it, i.e., a component consists of all variables that belong to its corresponding subproblem.

Each OCC cycle is mainly divided into two parts. The common operations of both parts are presented in Pseudocode 2 and Pseudocode 3. At first, we optimize components for the number of iterations that stem from the computational resource allocation

Pseudocode 1 The general OCC procedure

```
   input: f: an optimization problem, Θ: the interaction matrix,
   optimizer: a component optimizer
   output: x*: the best solution found by OCC
1: OC ← create overlapping components using Θ
2: x* ← create a random solution
3: Iters ← assign 1 to each component in OC
4: Contrs ← assign 0 to each component in OC
5: while stop condition is not met do
6:     OC ← shuffle OC
7:     ItersToAdd ← assign 0 to each component in OC
8:     for c in OC do
9:         impr ← false
10:        for i from 1 to min{Iters[c], |c|} do
11:            if Optimize(c, f, optimizer, x*, Contrs) then
12:                impr ← true
13:            end if
14:        end for
15:        if impr then
16:            UpdateIterCounters(c, Iters, ItersToAdd)
17:        end if
18:    end for
19:    AOC ← create an award list of OC based on Contrs
20:    AOC ← shuffle AOC
21:    for c in AOC do
22:        impr ← Optimize(c, f, optimizer, x*, Contrs)
23:        if impr then
24:            UpdateIterCounters(c, Iters, ItersToAdd)
25:        end if
26:    end for
27:    for c in OC do
28:        Iters[c] ← Iters[c] + ItersToAdd[c]
29:    end for
30: end while
31: return x*
```

Pseudocode 2 The optimization of a component for one iteration

```
   input: c: a component to be optimized, f: an optimization problem,
   optimizer: a component optimizer, x*: the best solution found so far,
   Contrs: accumulated contributions of components
   output: impr: has the best solution found so far been updated
1: function Optimize(c, f, optimizer, x*, Contrs)
2:     impr ← false
3:     x*_c ← run optimizer for c with context x*
4:     if f(x*_c) < f(x*) then
5:         Contrs[c] = 0.5 · Contrs[c] + 0.5 · (f(x*) − f(x*_c))
6:         x* ← x*_c
7:         impr ← true
8:     else
9:         Contrs[c] = 0.5 · Contrs[c]
10:    end if
11:    return impr
12: end function
```

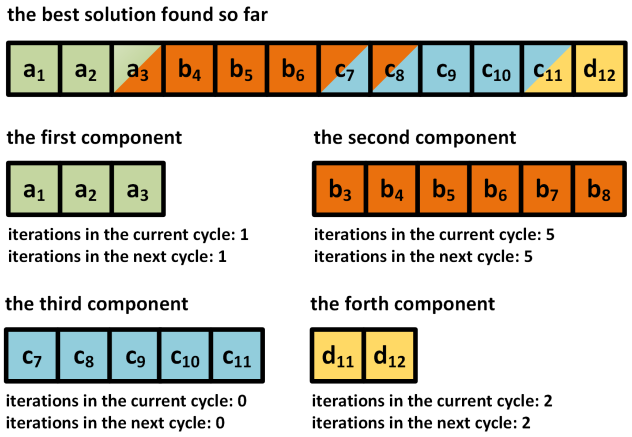
Pseudocode 3 Updating the number of iterations in the next cycle

```
   input: c: a component after the successful optimization, Iters:
   the number of iterations for which the component optimizers
   are run during the single cycle, ItersToAdd: the number of
   iterations to increase Iters counter at the end of the cycle
1: function UpdateIterCounters(c, Iters, ItersToAdd)
2:     AC ← get adjacent components to c
3:     for ac in AC do
4:         ItersToAdd[ac] ← ItersToAdd[ac] + 1
5:     end for
6:     Iters[c] = max{Iters[c] − 2, 0}
7:     ItersToAdd[c] = 0
8: end function
```

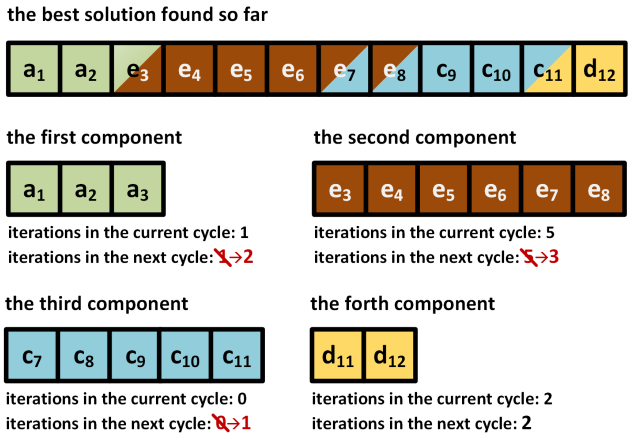
mechanism (Pseudocode 1: lines 6–18). Then, these components that influence the fitness improvements the most are selected to run the component optimizers for one more iteration. The other part is adopted from CBCCO [7] (Pseudocode 1: lines 19–26). The order of optimizing components for both parts is always random (lines 6 and 20). During the first part of the first OCC cycle, each component is optimized for one iteration (Pseudocode 1: lines 3, 10, and 11). If after optimizing component c the best individual found so far denoted as $\boldsymbol{x}^*$ is updated (Pseudocode 1: lines 11–13; Pseudocode 2: lines 4–8), we search for adjacent components to component c (Pseudocode 1: lines 15–17; Pseudocode 3: line 2). For each adjacent component, we increase the number of iterations during the next OCC cycle by 1 (Pseudocode 3: line 4). Moreover, we decrease the number of iterations in the next OCC cycle for component c by 2 (Pseudocode 3: line 6) and reset the number of iterations that are already added by the adjacent components during a particular OCC cycle (Pseudocode 3: line 7). We believe that during a single OCC cycle, at most one related component increases *ItersToAdd* counter on average. Therefore, decreasing the number of iterations by 2 should lead to excluding the component from the next OCC cycles faster when the optimization of its adjacent components do not result in improving the best solution found so far.

To prevent a situation when components that influence the fitness the most do not take part in the optimization because the optimization of their adjacent components cannot lead to finding the new best solution, we select such components that should be awarded according to [7] (Pseudocode 1: line 19). To this end, we calculate accumulated contributions (Pseudocode 2: lines 5 and 9). For each component that is awarded, we run the component optimizer for one iteration (Pseudocode 1: line 22). The consequences of finding the new best solution are the same as during the first part of the OCC cycle (Pseudocode 1: lines 23–25).

At the end of each OCC cycle, for each component we sum the current number of iterations and the number of iterations that were added by its adjacent components (Pseudocode 1: lines 27–29). In the next OCC cycle during its first part, we run the component optimizer for the number of iterations equals this value or the size of the component (Pseudocode 1: line 10). When the component optimizer converges it should not waste too much computational resources.



(a) Before the optimization of the second component



(b) After the successful optimization of the second component

Figure 2: The exemplary application of the new computational resource allocation mechanism.

The main idea of the new computational resource allocation mechanism is presented in Figure 2. The considered overlapping optimization problem is 12-dimensional, and it consists of four subproblems. Variables x_3, x_7, x_8, and x_{11} are shared variables. Thus, OCC creates four overlapping components. Figure 2a shows the state of OCC at the beginning of one of its cycles. The best solution found so far is $[a_1, a_2, a_3, b_4, b_5, b_6, c_7, c_8, c_9, c_{10}, c_{11}, d_{12}]$. For each component, we report the last solution that was injected into the global best and the number of iterations of their component optimizers in the current and the next OCC cycle. The second component is selected to be optimized first. After its successful optimization, i.e., the new best solution was found, the number of iterations in the next OCC cycle of the second component and its adjacent components is updated (Figure 2b).

4 EXPERIMENTS AND RESULTS ANALYSIS

This section presents the experimental results and their analysis. There was one main objective of our research. We would like to verify whether a CC framework after introducing overlapping components and the new computational resource allocation mechanism can solve overlapping LSGO problems more effectively. To check the effectiveness of our proposition, we compare OCC with CBCCO [7], CBCCOR, CC-RDG3 [26], and SHADE-ILS [17]. CBCCO and CC-RDG3 were shown to be effective in solving overlapping LSGO problems. CBCCOR is a modification of CBCCO that, to the best of our knowledge, was not proposed yet. In CBCCOR, components are optimized in random order. SHADE-ILS is the state-of-the-art LSGO-dedicated optimization method. Contrary to OCC, CBCCO, CBCCOR, and CC-RDG3, SHADE-ILS is a hybrid optimization method and does not utilize information about a possible problem structure provided by a decomposition strategy.

4.1 Test problems

Functions from IEEE CEC'2013 special session on LSGO [13] are used as benchmarks by many up-to-date papers considering LSGO [2, 11, 15, 25, 26]. However, only three functions are overlapping in this set: f_{12}, f_{13}, and f_{14}. We extend functions f_{13} and f_{14} as in [26] to increase the number of considered functions from 3 to 21.

Functions f_{13} and f_{14} consist of 20 overlapping subfunctions. Their adjacent subfunctions share $m = 5$ decision variables. In our experiments we consider $m \in \{1, ..., 10\}$ for both functions as in [26]. The sum of the number of variables of 20 subfunctions equals 1000. Due to the existence of shared variables, the number of variables of a function whose adjacent subfunctions share m variables is $n = 1000 - 19m$. f_{13} is an overlapping function with conforming subfunctions whereas f_{14} contains conflicting subfunctions [7, 11, 13, 26]. When two adjacent subfunctions are conforming, the optimal values of their shared variables are the same for both. In the case of two conflicting subfunctions, the optimal values of their shared variables may differ. Let us denote the conforming functions as $f_{o,m}$ and the conflicting functions as $f_{l,m}$ [26], where $m \in \{1, ..., 10\}$. Since we use almost the same set of test problems as in [26], the only exception is f_{12}, we also set the computation budget to $3 \cdot 10^6$ FFEs and repeat 40 times each experiment as well.

4.2 Optimization methods' setup

We used source code provided by their authors for each considered optimization method. The implementation of CBCCO was supported in C++[1]. We implemented CBCOOR by changing the original source code slightly. CC-RDG3 employs CMA-ES [5] as a component optimizer. CC-RDG3 was mainly written in Python because the Python implementation of CMA-ES[2] was shown to be the most efficient [1]. We executed RDG3 using the original source code written in MATLAB[3] and passed the decomposition results to CC. For SHADE-ILS, we used its Python source code[4]. Our proposition (OCC) was written in C++[5].

CBCOO, CBCOOR, and OCC require the interaction matrix of a considered optimization problem. We use the ideal decomposition matrix to mitigate the influence of the decomposition accuracy

[1] https://github.com/Flyki/Large-Scale-Overlapping-Optimization
[2] https://github.com/CMA-ES/pycma
[3] https://bitbucket.org/yuans/rdg3
[4] https://github.com/dmolina/shadeils
[5] https://github.com/kommar/OCC

on the effectiveness of these three CC frameworks [7]. The FFE-based stop condition is considered in our experiments [7, 11, 15, 17, 22, 26]. To ensure that using the ideal decomposition matrix does not discriminate against the other optimization methods, we decrease the computation budget for CBCOO, CBCOOR, and OCC. Nowadays, the ideal interaction matrix for a considered problem can be achieved only by decomposition strategies that perform the interaction check for all pairs of variables [7, 22]. An example of such a decomposition strategy is Differential Grouping 2 (DG2) [22]. The accuracy of discovering the variable interactions by DG2 is high when CEC'2013 functions are considered. The cost of running DG2 is $n(n+1)/2+1$ FFEs, where n is the considered problem size. Thus, the computation budget for CBCOO, CBCOOR, and OCC is decreased by this value, e.g., the optimization of $f_{l,7}$ by OCC can take 2,623,721 FFEs maximally.

We adopt parameter values for CBCOO, CC-RDG3, and SHADE-ILS from papers in which they were proposed. The number of iterations of the shared variable allocation stage of CBCOO (N) for $f_{o,m}$ and $f_{l,m}$, where $m \in \{1, ..., 10\}$, is set to 100 [7]. For f_{12}, we set $N = 10$. Since CBCOOR is a slight modification of CBCOO, we use the same parameter values. RDG3 has two parameters, ϵ_s and ϵ_n. We use $\epsilon_s = 100$ and $\epsilon_n = 50$ [26]. In every CC-RDG3 iteration, CMA-ES is executed for one iteration for each of components [26]. For SHADE-ILS, its underlying differential evolution maintains the population of 100 individuals, and the initial step of MTS-LS1 is set to 20 [17]. SHADE-ILS restarts itself when for three consecutive iterations the improvement ratio is lower than 5% [17]. OCC requires providing a component optimizer. We use CMA-ES [5] with its recommended parameter values because it is employed by the other competing methods that utilize information about a possible problem structure provided by a decomposition strategy.

4.3 Results

In Table 1, we present the optimization results for all considered test functions. Suppose we state that one optimization method is better or worse than the other. In that case, we have confirmed that by performing unpaired Wilcoxon test with the Holm-Bonferroni correction method at the significance level of 5%. OCC outperforms the other optimization methods for almost all of the overlapping functions with conforming subfunctions ($f_{o,1}$–$f_{o,10}$). The only exception is function $f_{o,6}$ where optimization results obtained by OCC and CBCOO do not differ significantly. Overlapping components in OCC do not split at least one subproblem. Consequently, all of the interacting variables of such subproblems are optimized simultaneously. Due to conforming subfunctions, the optimal value of each shared variable is the same for all of its related subfunctions. Thus, forcing finding the new best solution alternately by adjacent components is beneficial because the optimization of any part of the global solution is not neglected. Then, the component optimizer can search for new solutions consisting of the shared variable values that are around values already found by optimizers of its adjacent components, which should be of good quality at this optimization stage. Moreover, rewarding multiple best components leads to the faster convergence of OCC [7], because they are not blocked by other components. If a component influences global fitness the most but its related component cannot find the new best solution, OCC would seldomly optimize it when OCC did not reward best components. When the overlapping functions with conflicting subfunctions are considered ($f_{l,1}$–$f_{l,10}$), OCC is significantly better than CC-RDG3 and SHADE-ILS. OCC performs almost the same as CBCCO and CBCCOR. Overlapping components are not as suitable for conflicting subproblems as for conforming ones. Components that influence the global fitness more, dominate their less influential adjacent components. The optimization of such dominated components focuses on adjusting the dominating components (including shared variables) instead of searching for regions that are promising for them. For f_{12}, OCC is the worst proposition. f_{12} is a function that consists of 1000 variables and contains 999 subfunctions. Morover, 998 of its decision variables are shared. The main reason for lower quality results is probably the lack of joining too small components into larger ones as CBCOO and CBCOOR do. The results presented in Table 1 show also that the order of components to optimize in CBCCO is important for $f_{o,1}$–$f_{o,10}$.

OCC introduces the new computational resource allocation mechanism for overlapping components and adopts rewarding the best components from CBCOO [7]. To verify how they affect the effectiveness of OCC, we conducted experiments for three different OCC variants. They are presented in Table 2. In Table 3, we report the optimization results for the same test functions. The significance of the results have been confirmed by unpaired Wilcoxon test with the Holm-Bonferroni correction method at the significance level of 5%. The quality of results mainly increased after enabling the new computational resource allocation mechanism (compare OCC_3 and OCC_1 or OCC and OCC_2). However, OCC is significantly better than OCC_3 for the overlapping functions with conforming subfunctions ($f_{o,1}$–$f_{o,10}$) due to rewarding the best components. For the overlapping functions with conflicting subfunctions ($f_{l,1}$–$f_{l,10}$) such a mechanism does not influence the results significantly.

OCC behaves differently while optimizing overlapping functions of different types (with conforming subfuntions or with conflicting ones). In Figure 3, we present two histograms. The data was collected during all OCC runs for functions $f_{o,1}$–$f_{o,10}$ and $f_{l,1}$–$f_{l,10}$. We counted each iteration of a component optimizer that led to finding the new best solution in order to show how many times each component optimizer found the new best solution after all OCC cycles. Note that for the overlapping functions with conforming subfunctions, the distribution is left-skewed whereas a right-skewed distribution is reported for the other considered functions. This observation may be useful to distinguish conforming and conflicting subproblems from each other in black-box optimization.

5 CONCLUSION

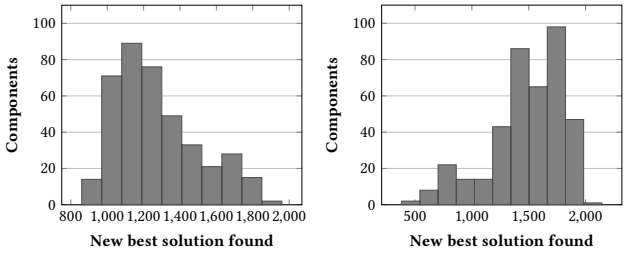
In this paper, we propose a new CC framework namely OCC. It differs from other CC frameworks proposed so far because OCC creates overlapping components instead of disjoint ones. Its overlapping components can be effectively optimized by utilizing the new computational resource allocation mechanism. In our experiments, OCC outperforms CBCOO, CBCOOR, CC-RDG3, and SHADE-ILS for the overlapping functions with conforming subfunctions. When the other overlapping functions are considered, OCC is usually

Table 1: Average best fitness values and OCC's number of wins, ties, and losses against the other considered optimizers

Fun	CBCCO	CBCCOR	CC-RDG3	SHADE-ILS	OCC
$f_{o,1}$	3.71E+04 ± 4.53E+04 ↑	1.60E+06 ± 2.55E+06 ↑	1.47E+04 ± 1.02E+04 ↑	1.23E+06 ± 9.37E+05 ↑	**2.42E+02** ± 1.22E+03
$f_{o,2}$	1.11E+04 ± 1.64E+04 ↑	1.34E+06 ± 1.74E+06 ↑	1.46E+04 ± 1.76E+04 ↑	2.47E+05 ± 9.97E+04 ↑	**8.87E+01** ± 3.53E+02
$f_{o,3}$	1.04E+04 ± 1.40E+04 ↑	3.36E+06 ± 9.25E+06 ↑	2.51E+03 ± 1.58E+03 ↑	1.98E+06 ± 1.31E+06 ↑	**2.55E+02** ± 9.02E+02
$f_{o,4}$	5.18E+03 ± 9.76E+03 ↑	5.18E+05 ± 9.49E+05 ↑	5.11E+03 ± 2.36E+03 ↑	5.03E+05 ± 3.78E+05 ↑	**8.27E+01** ± 8.46E+01
$f_{o,5}$	3.68E+03 ± 6.55E+03 ↑	5.64E+05 ± 8.83E+05 ↑	6.90E+03 ± 2.20E+03 ↑	9.11E+05 ± 6.61E+05 ↑	**2.40E+02** ± 2.97E+02
$f_{o,6}$	**6.23E+02** ± 1.01E+03	3.92E+04 ± 7.10E+04 ↑	1.68E+03 ± 9.10E+02 ↑	8.57E+05 ± 5.29E+05 ↑	**3.37E+02** ± 7.46E+02
$f_{o,7}$	9.84E+02 ± 1.20E+03 ↑	2.69E+05 ± 3.68E+05 ↑	5.44E+04 ± 4.17E+04 ↑	1.12E+06 ± 9.84E+05 ↑	**2.67E+02** ± 2.24E+02
$f_{o,8}$	2.57E+03 ± 1.68E+03 ↑	7.15E+03 ± 7.00E+03 ↑	4.47E+05 ± 5.09E+05 ↑	3.14E+05 ± 1.21E+05 ↑	**1.01E+03** ± 1.29E+03
$f_{o,9}$	2.36E+03 ± 1.37E+03 ↑	7.55E+03 ± 5.99E+03 ↑	1.46E+03 ± 6.19E+02 ↑	1.32E+06 ± 9.22E+05 ↑	**7.88E+02** ± 5.77E+02
$f_{o,10}$	1.93E+03 ± 9.52E+02 ↑	8.14E+03 ± 1.36E+04 ↑	9.85E+04 ± 6.96E+04 ↑	6.64E+05 ± 5.85E+05 ↑	**5.41E+02** ± 5.61E+02
$f_{l,1}$	**7.84E+05** ± 1.26E+04	8.43E+05 ± 7.63E+04 ↑	8.09E+05 ± 2.83E+04 ↑	3.83E+06 ± 1.42E+06 ↑	**7.82E+05** ± 2.32E+04
$f_{l,2}$	**1.09E+07** ± 2.96E+05	**1.09E+07** ± 2.73E+05	1.12E+07 ± 3.01E+05 ↑	1.39E+07 ± 1.24E+06 ↑	**1.08E+07** ± 3.00E+05
$f_{l,3}$	**1.03E+07** ± 9.22E+04	**1.03E+07** ± 6.79E+04	1.18E+07 ± 4.99E+05 ↑	1.31E+07 ± 1.51E+06 ↑	**1.03E+07** ± 1.22E+05
$f_{l,4}$	**1.09E+07** ± 3.71E+05	**1.09E+07** ± 3.34E+05	1.15E+07 ± 5.85E+05 ↑	1.35E+07 ± 1.14E+06 ↑	**1.11E+07** ± 6.77E+05
$f_{l,5}$	**4.43E+06** ± 5.52E+04	**4.43E+06** ± 4.87E+04	5.33E+06 ± 2.60E+05 ↑	7.39E+06 ± 1.08E+06 ↑	**4.43E+06** ± 1.07E+05
$f_{l,6}$	1.12E+08 ± 1.73E+06 ↑	1.12E+08 ± 1.48E+06 ↑	1.13E+08 ± 1.33E+06 ↑	1.17E+08 ± 2.19E+06 ↑	**1.11E+08** ± 1.28E+06
$f_{l,7}$	**1.60E+09** ± 8.68E+07	**1.58E+09** ± 5.14E+07	**1.59E+09** ± 8.23E+07	**1.58E+09** ± 7.64E+07	**1.59E+09** ± 7.56E+07
$f_{l,8}$	**2.03E+07** ± 8.77E+05 ↓	**2.05E+07** ± 1.02E+06	2.19E+07 ± 1.31E+06 ↑	2.56E+07 ± 1.89E+06 ↑	2.11E+07 ± 1.19E+06
$f_{l,9}$	**9.97E+07** ± 2.65E+06 ↓	**1.00E+08** ± 2.97E+06 ↓	1.05E+08 ± 4.39E+06	1.05E+08 ± 4.00E+06	1.07E+08 ± 5.42E+06
$f_{l,10}$	**7.95E+07** ± 1.46E+06	**7.99E+07** ± 2.40E+06	8.13E+07 ± 1.69E+06 ↑	8.34E+07 ± 1.63E+06 ↑	**7.92E+07** ± 1.62E+06
f_{12}	9.86E+02 ± 4.03E+01 ↓	9.92E+02 ± 5.80E+01 ↓	9.91E+02 ± 6.45E+00 ↓	**3.49E+01** ± 1.31E+02 ↓	1.26E+03 ± 1.23E+02
w/t/l	10/8/3	12/7/2	18/2/1	18/2/1	-/-/-

Table 2: Descriptions of other OCC variants

Variant	Description
OCC_1	OCC **without** the new computational resource allocation mechanism and **without** rewarding the best components (component optimizers are run for one iteration in each OCC_1 cycle)
OCC_2	OCC **without** the new computational resource allocation mechanism (component optimizers are run for one iteration in the first part of each OCC_2 cycle)
OCC_3	OCC **without** rewarding the best components



(a) Conforming subfunctions **(b) Conflicting subfunctions**

Figure 3: The number of new best solutions found during the whole component optimization process.

better than or similar to the other optimization methods that are taken into account in this paper.

Nevertheless, the effectiveness of OCC can be improved in the future, e.g., by joining too small components into larger ones. Finally, new mechanisms are required to effectively solve overlapping problems with conflicting subproblems. Especially when we will be able to classify a considered optimization problem as an overlapping problem with conforming subproblems or with conflicting subproblems.

ACKNOWLEDGMENTS

This work was supported by the Polish National Science Centre (NCN) under Grant 2020/38/E/ST6/00370 and Australian Research Council Discovery Grant (DP190101271).

REFERENCES

[1] Rafał Biedrzycki. 2019. On Equivalence of Algorithm's Implementations: The CMA-ES Algorithm and Its Five Implementations. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion* (Prague, Czech Republic) *(GECCO '19)*. 247–248.

[2] Minyang Chen, Wei Du, Yang Tang, Yaochu Jin, and Gary G. Yen. 2023. A Decomposition Method for Both Additively and Nonadditively Separable Problems. *IEEE Transactions on Evolutionary Computation* 27, 6 (2023), 1720–1734.

[3] Wei-Neng Chen, Ya-Hui Jia, Feng Zhao, Xiao-Nan Luo, Xing-Dong Jia, and Jun Zhang. 2019. A Cooperative Co-Evolutionary Approach to Large-Scale Multisource Water Distribution Network Optimization. *IEEE Transactions on Evolutionary Computation* 23, 5 (2019), 842–857.

[4] A. P. Engelbrecht, P. Bosman, and K. M. Malan. 2022. The influence of fitness landscape characteristics on particle swarm optimisers. *Natural Computing* 21, 2 (2022), 335–345.

[5] Nikolaus Hansen and Andreas Ostermeier. 2001. Completely Derandomized Self-Adaptation in Evolution Strategies. *Evolutionary Computation* 9, 2 (2001),

Table 3: Average best fitness values and OCC's number of wins, ties, and losses against the other OCC variants

Fun	OCC_1	OCC_2	OCC_3	OCC
$f_{o,1}$	3.38E+06 ± 3.16E+06 ↑	1.37E+05 ± 1.25E+05 ↑	1.85E+03 ± 2.82E+03 ↑	**2.42E+02** ± 1.22E+03
$f_{o,2}$	5.84E+06 ± 1.00E+07 ↑	6.92E+05 ± 6.55E+05 ↑	2.66E+03 ± 4.35E+03 ↑	**8.87E+01** ± 3.53E+02
$f_{o,3}$	6.79E+06 ± 8.12E+06 ↑	1.39E+06 ± 1.49E+06 ↑	4.76E+03 ± 6.56E+03 ↑	**2.55E+02** ± 9.02E+02
$f_{o,4}$	6.14E+06 ± 7.98E+06 ↑	1.90E+06 ± 1.31E+06 ↑	4.43E+03 ± 9.87E+03 ↑	**8.27E+01** ± 8.46E+01
$f_{o,5}$	4.30E+06 ± 2.54E+06 ↑	2.02E+06 ± 1.33E+06 ↑	4.15E+03 ± 4.25E+03 ↑	**2.40E+02** ± 2.97E+02
$f_{o,6}$	2.27E+06 ± 1.91E+06 ↑	1.21E+06 ± 7.62E+05 ↑	2.14E+03 ± 2.43E+03 ↑	**3.37E+02** ± 7.46E+02
$f_{o,7}$	4.56E+06 ± 3.17E+06 ↑	2.21E+06 ± 2.04E+06 ↑	4.26E+03 ± 9.82E+03 ↑	**2.67E+02** ± 2.24E+02
$f_{o,8}$	5.64E+06 ± 3.10E+06 ↑	3.66E+06 ± 1.61E+06 ↑	6.73E+03 ± 6.35E+03 ↑	**1.01E+03** ± 1.29E+03
$f_{o,9}$	1.26E+07 ± 1.52E+07 ↑	5.35E+06 ± 3.32E+06 ↑	6.35E+03 ± 5.94E+03 ↑	**7.88E+02** ± 5.77E+02
$f_{o,10}$	5.22E+06 ± 2.92E+06 ↑	3.15E+06 ± 1.59E+06 ↑	3.30E+03 ± 3.25E+03 ↑	**5.41E+02** ± 5.61E+02
$f_{l,1}$	9.06E+05 ± 1.28E+05 ↑	8.10E+05 ± 2.49E+04 ↑	**7.82E+05** ± 2.26E+04	**7.82E+05** ± 2.32E+04
$f_{l,2}$	1.11E+07 ± 4.77E+05 ↑	**1.09E+07** ± 3.46E+05	**1.08E+07** ± 3.07E+05	**1.08E+07** ± 3.00E+05
$f_{l,3}$	1.05E+07 ± 1.52E+05 ↑	1.04E+07 ± 1.05E+05 ↑	**1.03E+07** ± 1.00E+05	**1.03E+07** ± 1.22E+05
$f_{l,4}$	1.14E+07 ± 7.35E+05 ↑	1.11E+07 ± 5.49E+05 ↑	**1.11E+07** ± 6.25E+05	**1.11E+07** ± 6.77E+05
$f_{l,5}$	4.62E+06 ± 1.43E+05 ↑	4.56E+06 ± 1.01E+05 ↑	**4.42E+06** ± 9.41E+04	**4.43E+06** ± 1.07E+05
$f_{l,6}$	1.11E+08 ± 1.45E+06 ↑	**1.11E+08** ± 1.57E+06	**1.10E+08** ± 1.69E+06	**1.11E+08** ± 1.28E+06
$f_{l,7}$	**1.62E+09** ± 1.18E+08	**1.62E+09** ± 1.19E+08	**1.62E+09** ± 7.26E+07	**1.59E+09** ± 7.56E+07
$f_{l,8}$	2.17E+07 ± 1.03E+06 ↑	2.27E+07 ± 2.46E+06 ↑	**2.19E+07** ± 1.97E+06	**2.11E+07** ± 1.19E+06
$f_{l,9}$	**1.07E+08** ± 5.24E+06	**1.08E+08** ± 5.46E+06	**1.08E+08** ± 4.86E+06	**1.07E+08** ± 5.42E+06
$f_{l,10}$	8.02E+07 ± 1.95E+06 ↑	8.02E+07 ± 2.10E+06 ↑	**7.92E+07** ± 1.24E+06	**7.92E+07** ± 1.62E+06
f_{12}	**1.14E+03** ± 1.12E+02 ↓	**1.18E+03** ± 8.53E+01 ↓	1.51E+03 ± 1.13E+03	1.26E+03 ± 1.23E+02
w/t/l	18/2/1	16/4/1	10/11/0	-/-/-

159–195.

[6] Ya-Hui Jia, Yi Mei, and Mengjie Zhang. 2020. A Memetic Level-Based Learning Swarm Optimizer for Large-Scale Water Distribution Network Optimization. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference* (Cancún, Mexico) *(GECCO '20)*. 1107–1115.

[7] Ya-Hui Jia, Yi Mei, and Mengjie Zhang. 2022. Contribution-Based Cooperative Co-Evolution for Nonseparable Large-Scale Problems With Overlapping Subcomponents. *IEEE Transactions on Cybernetics* 52, 6 (2022), 4246–4259.

[8] Ya-Hui Jia, Yu-Ren Zhou, Ying Lin, Wei-Jie Yu, Ying Gao, and Lu Lu. 2019. A Distributed Cooperative Co-evolutionary CMA Evolution Strategy for Global Optimization of Large-Scale Overlapping Problems. *IEEE Access* 7 (2019), 19821–19834.

[9] Jun-Rong Jian, Zhi-Hui Zhan, and Jun Zhang. 2020. Large-scale evolutionary optimization: a survey and experimental comparative study. *International Journal of Machine Learning and Cybernetics* 11 (2020).

[10] Pascal Kerschke and Heike Trautmann. 2019. Automated Algorithm Selection on Continuous Black-Box Problems by Combining Exploratory Landscape Analysis and Machine Learning. *Evolutionary Computation* 27, 1 (2019), 99–127.

[11] Marcin Michal Komarnicki, Michal Witold Przewozniczek, Halina Kwasnicka, and Krzysztof Walkowiak. 2023. Incremental Recursive Ranking Grouping for Large-Scale Global Optimization. *IEEE Transactions on Evolutionary Computation* 27, 5 (2023), 1498–1513.

[12] Antonio LaTorre, Santiago Muelas, and José-María Peña. 2015. A comprehensive comparison of large scale global optimizers. *Information Sciences* 316 (2015), 517–549. Nature-Inspired Algorithms for Large Scale Global Optimization.

[13] Xiaodong Li, Ke Tang, Mohammmad Nabi Omidvar, Zhenyu Yang, and Kai Qin. 2013. Benchmark Functions for the CEC'2013 Special Session and Competition on Large-Scale Global Optimization. (2013).

[14] Lin-Yu Tseng and Chun Chen. 2008. Multiple trajectory search for Large Scale Global Optimization. In *2008 IEEE Congress on Evolutionary Computation (IEEE World Congress on Computational Intelligence)*. 3052–3059.

[15] Xiaoliang Ma, Zhitao Huang, Xiaodong Li, Lei Wang, Yutao Qi, and Zexuan Zhu. 2022. Merged Differential Grouping for Large-Scale Global Optimization. *IEEE Transactions on Evolutionary Computation* 26, 6 (2022), 1439–1451.

[16] Olaf Mersmann, Mike Preuss, and Heike Trautmann. 2010. Benchmarking Evolutionary Algorithms: Towards Exploratory Landscape Analysis. In *Parallel Problem Solving from Nature, PPSN XI*, Robert Schaefer, Carlos Cotta, Joanna Kołodziej, and Günter Rudolph (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 73–82.

[17] Daniel Molina, Antonio LaTorre, and Francisco Herrera. 2018. SHADE with Iterative Local Search for Large-Scale Global Optimization. In *2018 IEEE Congress on Evolutionary Computation (CEC)* (Rio de Janeiro, Brazil). IEEE Press, 1–8.

[18] José Luis Morales and Jorge Nocedal. 2011. Remark on "Algorithm 778: L-BFGS-B: Fortran Subroutines for Large-Scale Bound Constrained Optimization". 38, 1, Article 7 (2011), 4 pages.

[19] Mohammad Nabi Omidvar, Xiaodong Li, Yi Mei, and Xin Yao. 2014. Cooperative Co-Evolution With Differential Grouping for Large Scale Optimization. *IEEE Transactions on Evolutionary Computation* 18, 3 (2014), 378–393.

[20] Mohammad Nabi Omidvar, Xiaodong Li, and Ke Tang. 2015. Designing benchmark problems for large-scale continuous optimization. *Information Sciences* 316 (2015), 419–436. Nature-Inspired Algorithms for Large Scale Global Optimization.

[21] Mohammad Nabi Omidvar, Xiaodong Li, and Xin Yao. 2011. Smart Use of Computational Resources Based on Contribution for Cooperative Co-Evolutionary Algorithms. In *Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation* (Dublin, Ireland) *(GECCO '11)*. 1115–1122.

[22] Mohammad Nabi Omidvar, Ming Yang, Yi Mei, Xiaodong Li, and Xin Yao. 2017. DG2: A Faster and More Accurate Differential Grouping for Large-Scale Black-Box Optimization. *IEEE Transactions on Evolutionary Computation* 21, 6 (2017), 929–942.

[23] Mitchell A. Potter and Kenneth A. De Jong. 1994. A cooperative coevolutionary approach to function optimization. In *Parallel Problem Solving from Nature — PPSN III*, Yuval Davidor, Hans-Paul Schwefel, and Reinhard Männer (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 249–257.

[24] Michal Witold Przewozniczek and Marcin Michal Komarnicki. 2020. Empirical Linkage Learning. *IEEE Transactions on Evolutionary Computation* 24, 6 (2020), 1097–1111.

[25] Yuan Sun, Michael Kirley, and Saman K. Halgamuge. 2018. A Recursive Decomposition Method for Large Scale Continuous Optimization. *IEEE Transactions on Evolutionary Computation* 22, 5 (2018), 647–661.

[26] Yuan Sun, Xiaodong Li, Andreas Ernst, and Mohammad Nabi Omidvar. 2019. Decomposition for Large-scale Optimization Problems with Overlapping Components. In *2019 IEEE Congress on Evolutionary Computation (CEC)*. 326–333.

[27] R. Tanabe and A. Fukunaga. 2013. Success-history based parameter adaptation for Differential Evolution. In *2013 IEEE Congress on Evolutionary Computation*. 71–78.

[28] R. Tinós, D. Whitley, and F. Chicano. 2015. Partition crossover for pseudo-boolean optimization. In *Proceedings of the 2015 ACM Conference on Foundations of Genetic*

Algorithms XIII. 137–149.

[29] Ming Yang, Aimin Zhou, Changhe Li, Jing Guan, and Xuesong Yan. 2020. CCFR2: A more efficient cooperative co-evolutionary framework for large-scale global optimization. *Information Sciences* 512 (2020), 64 – 79.

[30] Zhi-Hui Zhan, Lin Shi, Kay Tan, and Jun Zhang. 2021. A survey on evolutionary computation for complex continuous optimization. *Artificial Intelligence Review* (2021).